High Level Designing
IAC Provisioning For Finance System

Course Name: DevOps

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrollment Number
01	**Mahima Patidar**	**EN22CS301569**
02	**Nikhil Rathore**	**EN22CS301650**
03	**Nitisha Kushwah**	**EN22CS301664**
04	**Nidhish Mehta**	**EN22CS301642**
05	**Moin Husain**	**EN22IT301612**

Group Name: D6 G12

Project Number: DO-23

Industry Mentor Name: Vaibhav

University Mentor Name: Akshay Saxena

Academic Year: 2025-2026

1. Introduction

1.1 Scope of the Document

This document describes the **High Level Design (HLD)** of the Infrastructure as Code (IaC) based provisioning system for a Finance Platform.
The purpose of this design is to define the overall system architecture, major components, and the interaction between them for automating infrastructure deployment and management.

The system aims to provide a **secure, scalable, and automated infrastructure provisioning mechanism** for finance-related applications by integrating containerization, orchestration, and continuous deployment tools.

1.2 Intended Audience

• **Project Supervisors**
To review the system architecture and evaluate whether the solution meets the project objectives.

• **DevOps Engineers**
To understand the automation framework used for infrastructure provisioning and deployment.

• **System Architects**
To analyze the architectural components, their interactions, and integration with container orchestration platforms.

1.3 System Overview

The Finance Infrastructure Automation System is designed to **automatically provision and manage infrastructure resources** required for financial applications.

The architecture integrates:

- Infrastructure as Code (IaC) tools
- Containerization technologies
- Continuous Integration and Deployment pipelines

The system ensures that financial microservices such as **payment processing, account management, billing, and transaction services** are deployed in a consistent and scalable environment.

Automation eliminates manual infrastructure setup and improves reliability, scalability, and deployment speed across development, testing, and production environments.

2. System Design

2.1 Application Design

The system follows a **Microservices-based architecture** where different finance services operate independently.

Each service runs inside a containerized environment and communicates with other services through defined interfaces.

The architecture includes the following layers:

- **Application Layer** – Financial microservices such as billing, transaction processing, and reporting.
- **Container Layer** – Docker containers that package applications and dependencies.
- **Orchestration Layer** – Kubernetes manages container deployment, scaling, and availability.
- **Automation Layer** – CI/CD tools manage the deployment lifecycle.

This layered approach improves system flexibility, maintainability, and scalability.

2.2 Process Flow

The high-level process flow of the system is as follows:

1. Developers create infrastructure configuration files and application code.
2. The code is stored in a version-controlled repository.
3. When code changes are pushed, the CI/CD system automatically detects the update.
4. The automation pipeline validates the infrastructure configuration.
5. Infrastructure provisioning tools create or update the required resources.
6. Containerized applications are deployed to the orchestration platform.
7. The deployed services become available to users or internal financial systems.

This automated workflow ensures faster deployment and reduces manual intervention.

2.3 Information Flow

The information flow begins with the **source code repository**, which stores infrastructure scripts and application code.

The automation platform retrieves the configuration and processes it through the deployment pipeline.

Infrastructure provisioning tools interpret the configuration and create the required resources on the target environment.

Once infrastructure resources are created, container orchestration platforms deploy and manage financial services.

A centralized state management system keeps track of the infrastructure resources and ensures synchronization between the defined configuration and the actual deployed infrastructure.

2.4 Components Design

Git Repository

Acts as the central storage for infrastructure scripts and application source code.
It ensures version control, collaboration, and change tracking.

CI/CD Pipeline

The automation engine responsible for building, testing, and deploying infrastructure and application components whenever changes occur.

Infrastructure Provisioning Tool

Responsible for creating and managing infrastructure resources based on defined configuration files.

Container Runtime Environment

Provides a standardized environment for running microservices using containerization technology.

Container Orchestration Platform

Manages container deployment, scaling, service discovery, and resource allocation across the infrastructure.

2.5 Key Design Considerations

Modularity

Infrastructure and application components are organized into separate modules to improve maintainability and reuse.

Scalability

The architecture supports dynamic scaling of services to handle increasing transaction volumes.

Reliability

The system ensures service availability through container orchestration and automated recovery mechanisms.

Automation

The entire infrastructure lifecycle is automated to reduce manual errors and improve operational efficiency.

2.6 API Catalogue

The system uses APIs to communicate with the infrastructure management platform.

Container Orchestration API

Provides endpoints for managing containers, deployments, and service configurations within the orchestration platform.

Automation API

Used by the CI/CD pipeline to trigger infrastructure provisioning and application deployment tasks.

---

3. Data Design

3.1 Data Model

The system maintains metadata related to infrastructure provisioning and deployment activities.

The key entities include:

- Infrastructure Modules
- Deployment Jobs
- Infrastructure Resources
- Financial Services
- Environment Configurations

These entities represent the logical structure required to manage infrastructure and application deployments.

3.2 Logical Relationships

1. **Job Triggers Module (1:N)**
   A single deployment job can execute multiple infrastructure modules.
2. **Job Creates State (1:1)**
   Each deployment job generates one infrastructure state record.
3. **Module Defines Resource (1:N)**
   Each infrastructure module may define multiple infrastructure resources.

4. **Resource Hosts Service (1:N)**
   A single infrastructure resource may host multiple financial services.
5. **Resource Belongs to Environment (N:1)**
   Multiple resources are associated with a specific environment such as development, testing, or production.

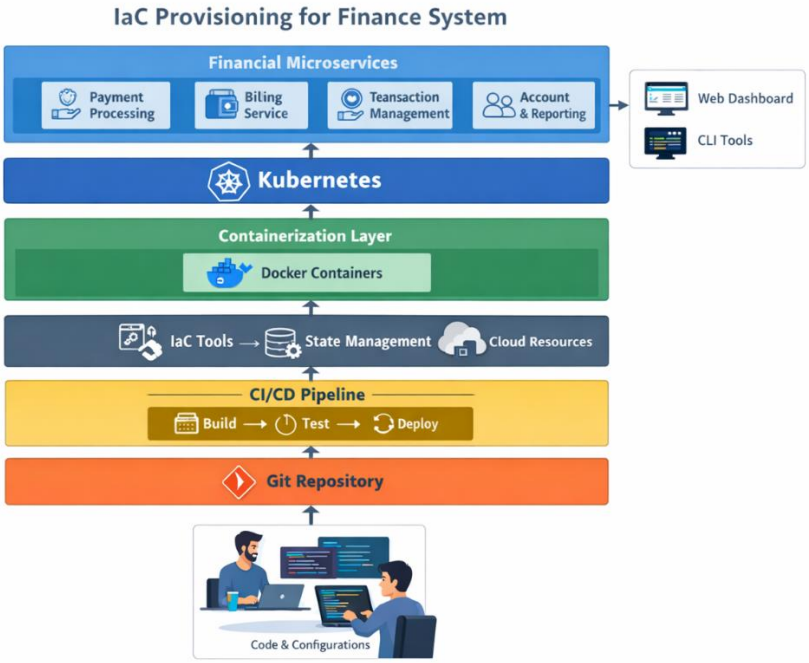

Figure 3.1 High Level Design Architecture

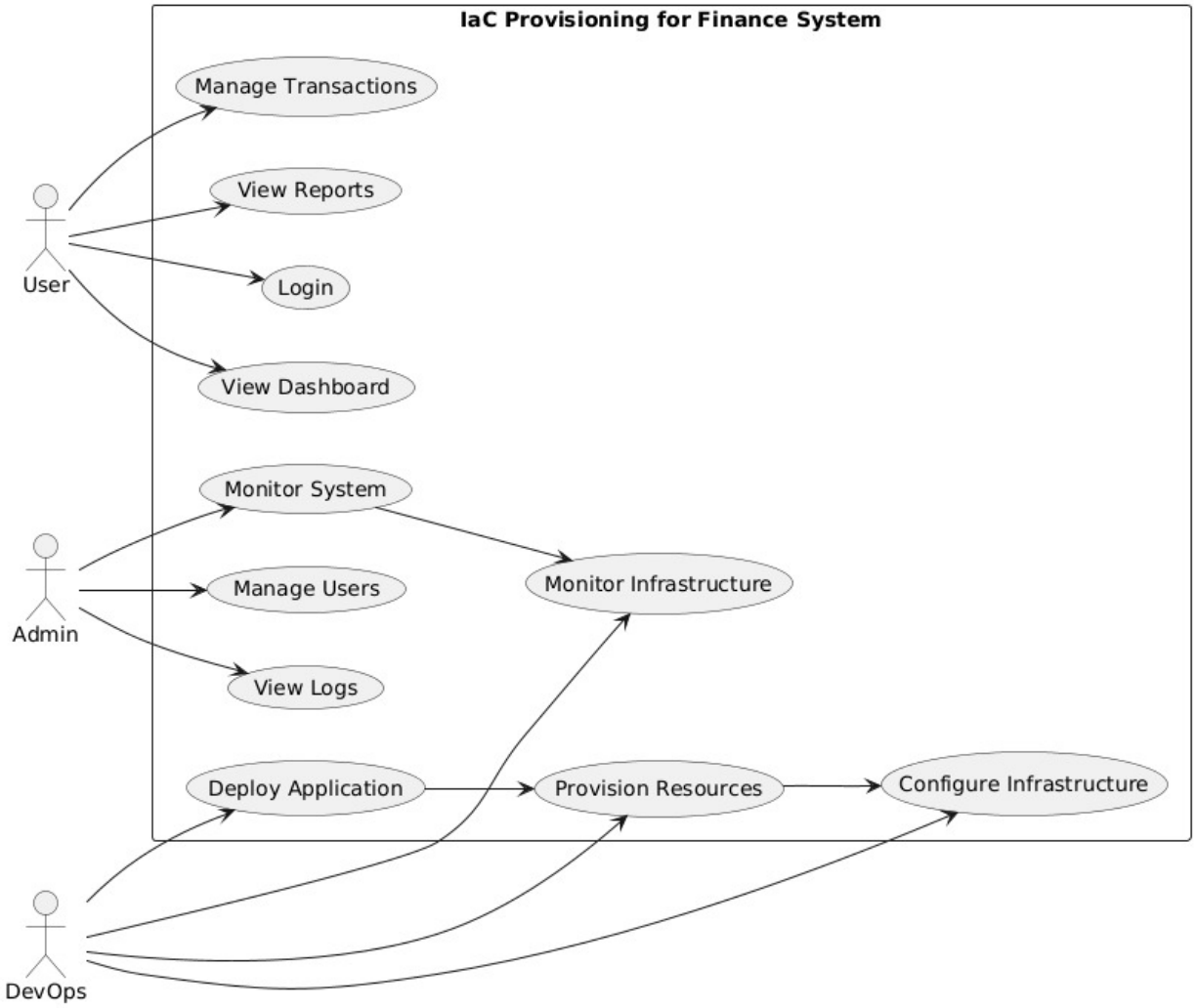

Fig 3.2 Use Case Diagram

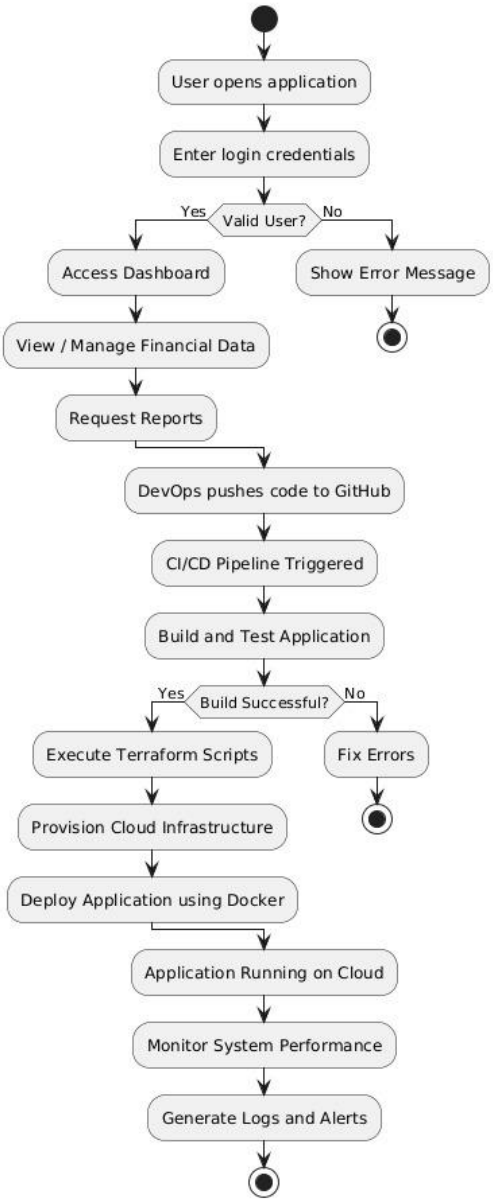

Fig 3.3 Activity Diagram

3.3 Data Access Mechanism

Infrastructure configuration and metadata are accessed through secure repositories and controlled storage systems.

Access control policies ensure that only authorized users can modify infrastructure configurations or deployment data.

3.4 Data Retention Policies

Deployment logs and infrastructure state information are stored for auditing and recovery purposes.

Historical records of infrastructure configurations are maintained to support rollback operations and disaster recovery procedures.

3.5 Data Migration

Infrastructure migration between environments is supported through configuration updates in the infrastructure provisioning tools.

This allows the system to deploy the same architecture across different environments such as local development platforms and cloud infrastructure providers.

4. Interfaces

4.1 User Interface

A web-based dashboard provides visibility into deployment pipelines, infrastructure status, and application health.

Administrators can monitor deployments, view logs, and manage infrastructure operations through this interface.

Command Line Interface (CLI)

Developers and DevOps engineers interact with the infrastructure using command-line tools to execute deployment commands, manage repositories, and update configurations.

---

5. State and Session Management

The system maintains infrastructure state information in a centralized storage mechanism.

This state file tracks all deployed infrastructure resources and helps the system determine what changes are required during future deployments.

State management ensures consistency between defined infrastructure configurations and the actual deployed environment.

---

6. Caching

Caching mechanisms are used to improve system performance and reduce deployment time.

Container build layers are cached to avoid rebuilding unchanged components.

Infrastructure tool plugins and dependencies are also cached locally to accelerate execution during repeated deployments.

7. Functional Requirements

7.1 User Authentication & Authorization

The system must provide secure access control for administrators managing infrastructure deployments.

Users must authenticate before performing operations such as triggering deployments or modifying configurations.

Authorization policies restrict access to sensitive infrastructure management operations.

7.2 Infrastructure Configuration Management

The system should support centralized management of infrastructure configuration parameters.

Administrators should be able to manage environment configurations and maintain deployment scripts in a structured repository.

Deployment metadata and system activity logs must be recorded for monitoring and auditing purposes.

7.3 Provisioning Control & Monitoring

Users should be able to initiate infrastructure provisioning through the management portal.

The system must provide monitoring capabilities to track deployment progress and resource status.

Infrastructure components should be installed and configured automatically during provisioning.

7.4 Finance Service Orchestration

Administrators must be able to manage the lifecycle of financial services deployed in the infrastructure.

This includes starting, stopping, and restarting services when required.

The system must maintain a mapping between infrastructure resources and the services deployed on them.

7.5 Dockerfile (Application Containerization)

Application components are packaged into container images to ensure consistency across development, testing, and production environments.

Containerization ensures portability and simplifies application deployment.

7.6 CI/CD Workflow Configuration

The CI/CD workflow automates the build, testing, and deployment of infrastructure and application components whenever code changes occur.

Automation ensures rapid delivery of updates while maintaining system stability.

7.7 CI/CD Strategy

Continuous Integration

Code updates are automatically validated and tested before deployment.

This ensures that infrastructure configurations and application code meet quality and security standards.

Continuous Deployment

Once validated, infrastructure changes and application updates are deployed automatically.

Container images are delivered to the container orchestration platform, where services are updated without affecting system availability.

---

8. Non-Functional Requirements

8.1 Security Aspects

The system must ensure that sensitive infrastructure credentials and configuration data are securely managed.

Access control, encrypted storage, and secure communication channels are implemented to protect the system.

8.2 Performance Aspects

Automation significantly reduces infrastructure setup time and improves operational efficiency.

The architecture ensures consistent deployment across multiple environments while maintaining high availability and performance.

---

9. References

Problem Statement: **IaC Provisioning for Finance System**
Project Group: **D6 – Group 12**

Project Number: **DO-23**

---

10. Github

https://github.com/nidhishmehta03/expense_tracker/blob/main/Dockerfile